

[RU](#) · [ZH](#)

OPERATION MANUAL



LCD-bitmap IDE

A desktop workbench for monochrome embedded LCD interfaces: screens, the navigation state machine, the physical control panel, runtime tags and firmware-ready export — kept in a single project file.

Software version 0.1.18 **Project format** .lcdproj, schema 6

Platforms Windows · Linux · macOS **Builds** Electron · Tauri

First Launch

The application needs no external services and makes no network calls: an entire project is one `.lcdproj` file.

- 1 Install the build for your OS from [GitHub Releases](#), or run the source with `npm run electron:dev`.
- 2 On the start screen, pick an interface language, then one of four entry points: open an existing `.lcdproj`, create a new project, restore an autosave, or open the bundled demo project.
- 3 For a first look, choose **Demo** — it already contains linked states, screens and transitions you can learn from safely.



Start screen. Interface language (EN / RU / ZH) and entry points: open a project, create a new one, restore an autosave, or open the demo.

The project autosaves to `localStorage` every few seconds. That is a crash safety net, not a substitute for an explicit **Save project** — save the file to disk yourself before closing the session.

Interface Overview

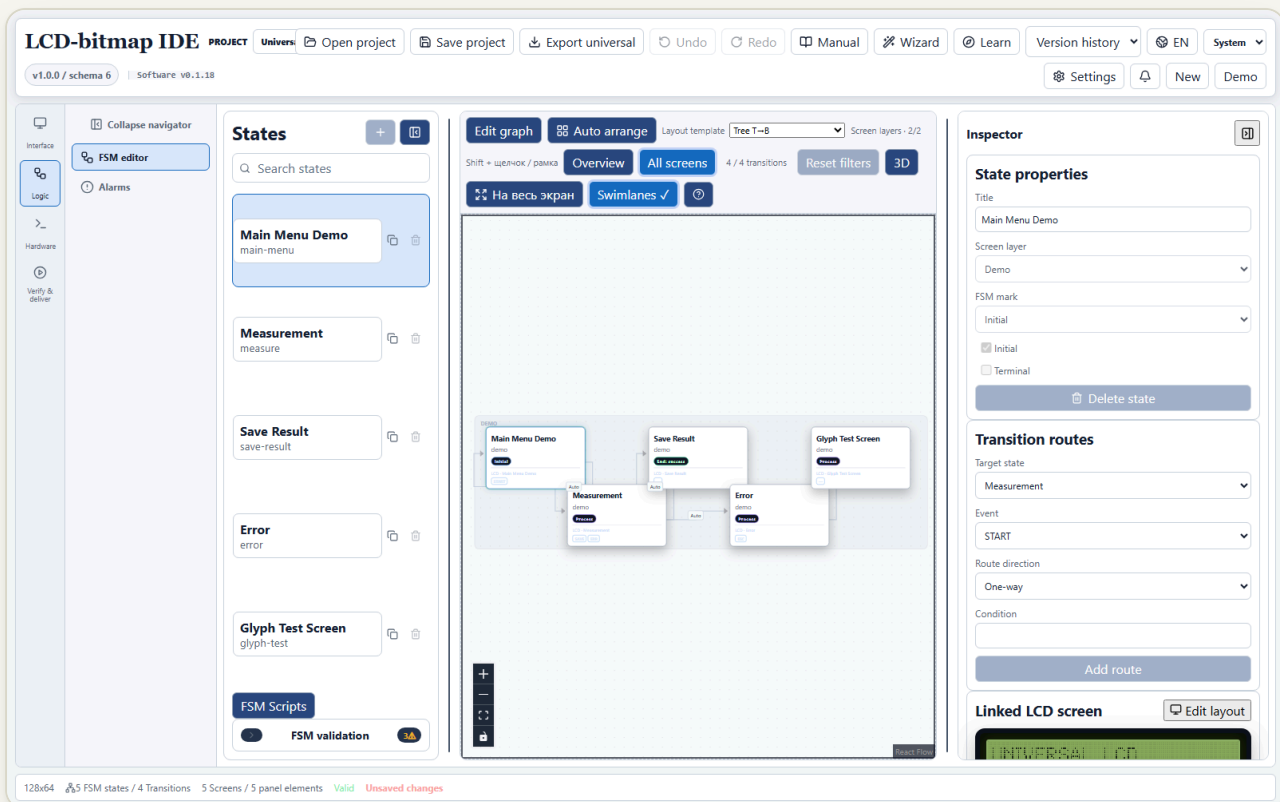
On the left is a compact navigator of workspaces grouped by purpose:

Interface (screens and the panel), **Logic** (FSM and alarms), **Hardware** (tags and procedures), **Verify & deliver** (simulation and firmware handoff). Every workspace has its own independently collapsible side panes.

- The `Collapse navigator` control (or `Ctrl+B`) collapses only the global navigator into a narrow icon rail — the current workspace's own side panels stay put.
- The status bar at the bottom always shows display size, FSM state/transition counts, screen and panel-element counts, plus validation status and unsaved-change state.
- The theme switch (Settings → Theme) supports light, dark and system-follow modes.

FSM Editor

The finite-state machine is the device's navigation map: a state corresponds to one LCD screen, a transition to a reaction to a button, timer or condition.

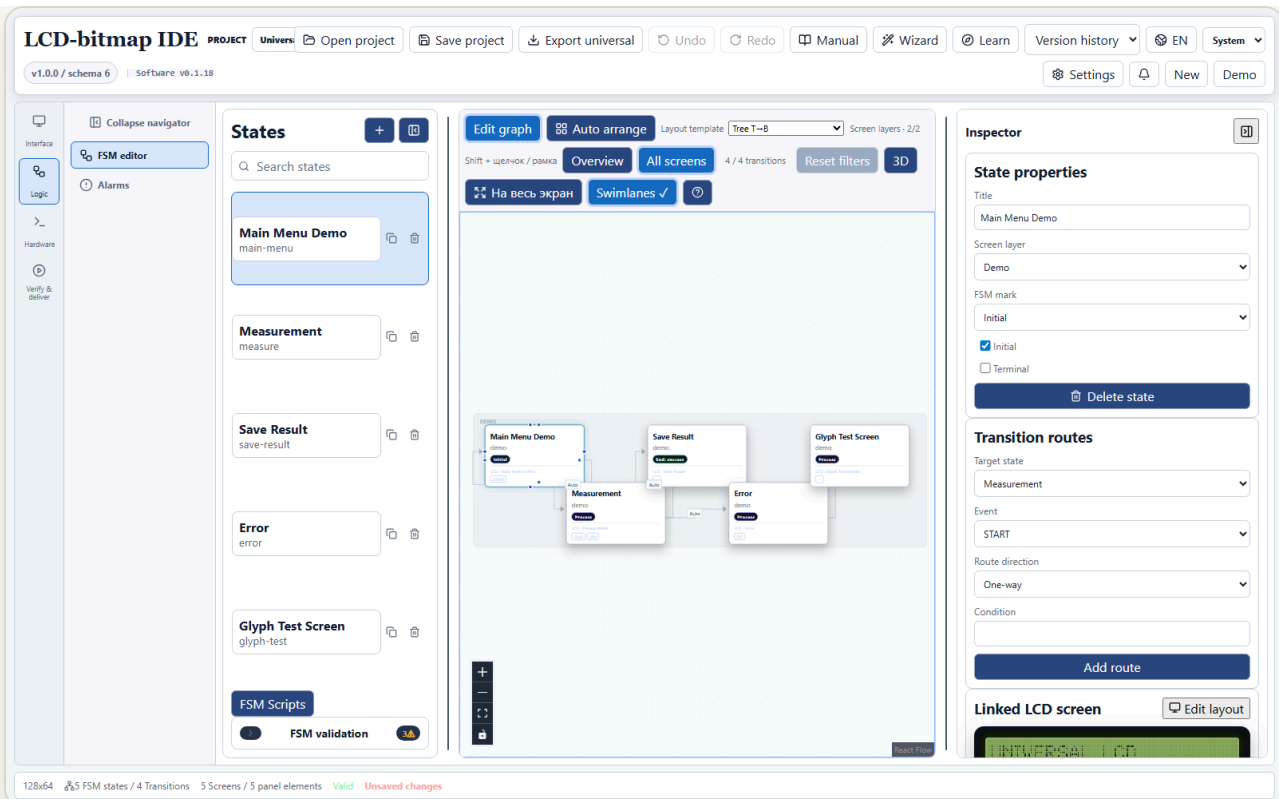


FSM graph in read-only mode. Left: the state catalog. Center: the transition graph. Right: the inspector for the selected item. Connections stay visible regardless of edit mode.

Edit mode

The graph opens *read-only* by default — a deliberate guard against accidentally changing live logic. Graph-mutating controls (adding a state, deleting, dragging) only become active once you explicitly turn on

Edit graph .



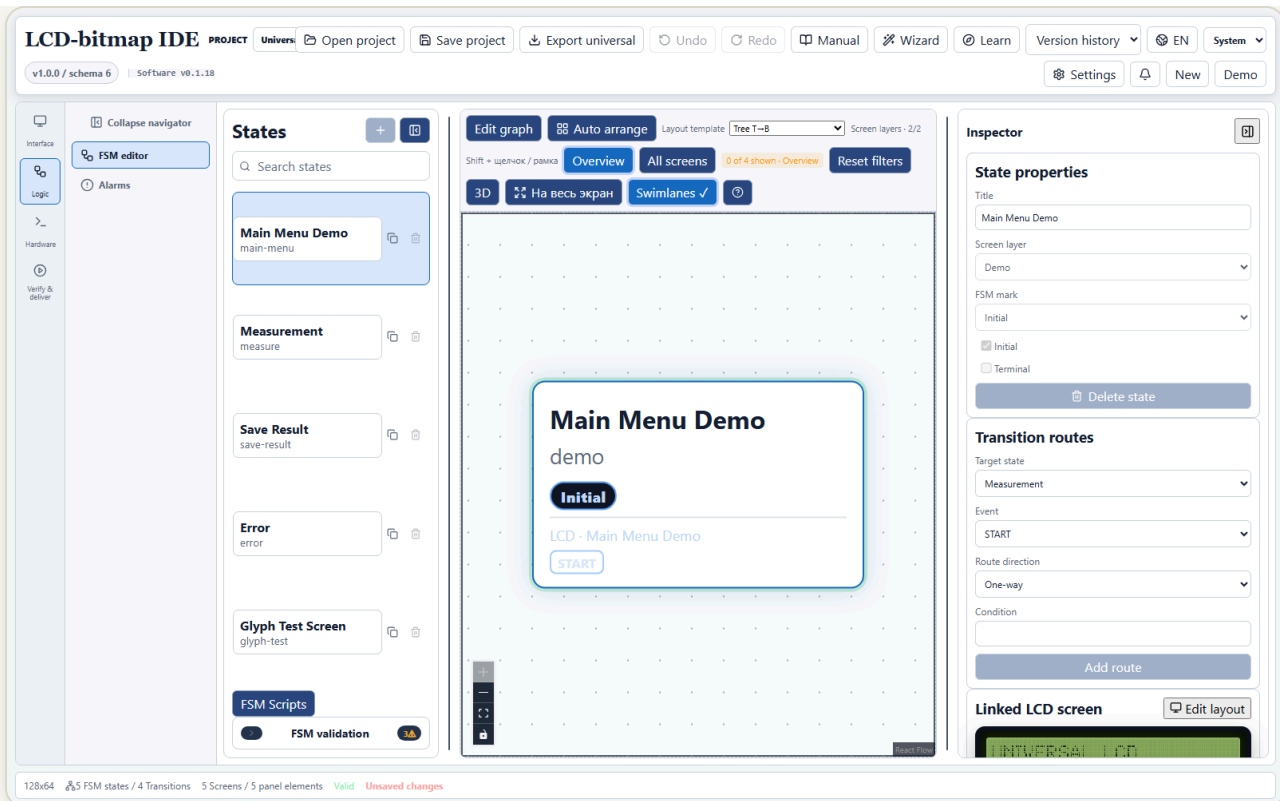
Edit mode on – the canvas outline turns blue; creating states and dragging nodes becomes available.

Creating a state and a transition

- 1 Turn on **Edit graph**, then click the + above the state list, or right-click empty canvas → **Add state**.
- 2 Name the state and, if needed, mark it **Initial** (entry point) or **Terminal** (end point).
- 3 To connect states, either drag between the connection handles on the nodes, or use the **Transition routes** section in the right-hand inspector — pick the target state and event without touching the canvas directly.
- 4 Link the state to an LCD screen via **Linked LCD screen** → **Edit layout** to jump straight into its pixel design.

Overview and filters

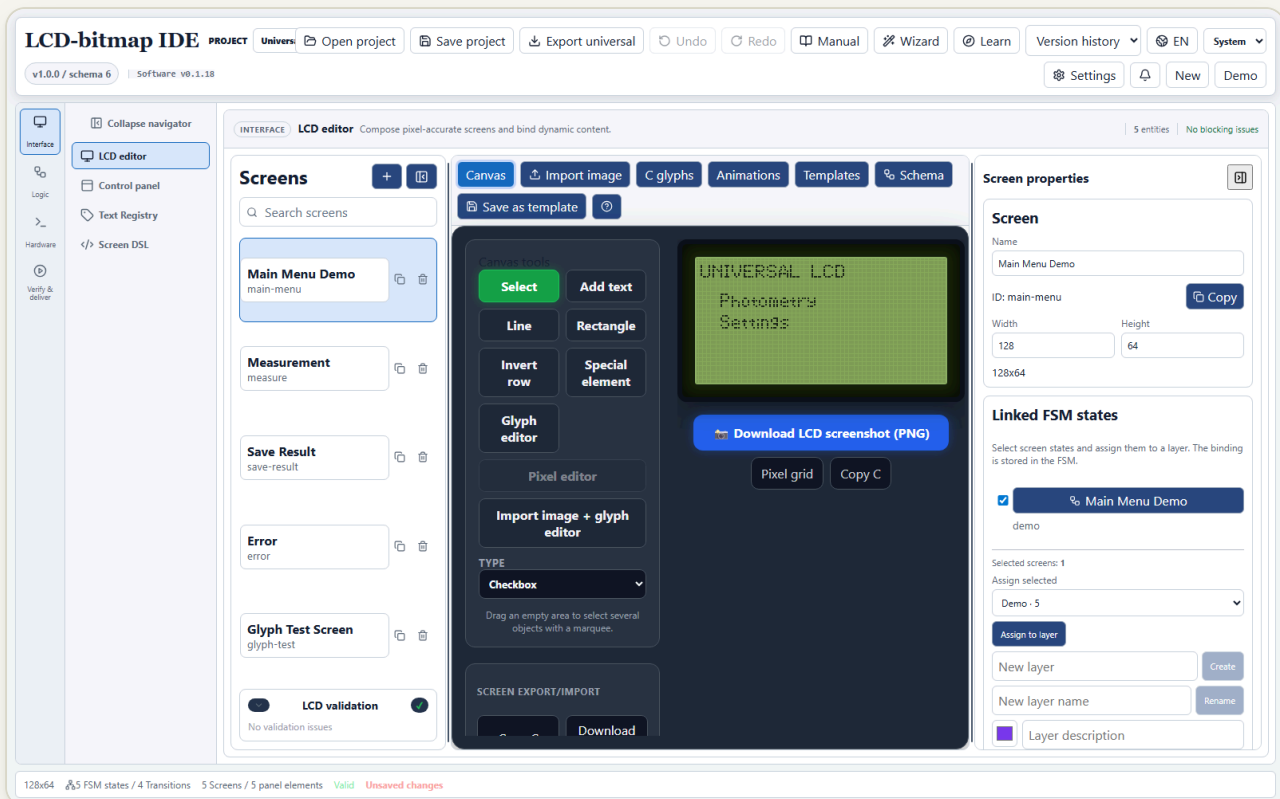
For a large graph, turn on **Overview** — it shows one entry point per subsystem instead of the full graph. Whenever Overview, a subsystem focus or a layer filter hides part of the transitions, the toolbar shows a “**shown / total**” counter and the reason — a one-click filter reset sits right next to it.



Overview mode groups states by subsystem and entry point – useful for navigating large projects.

LCD Screen Editor

Every screen is a pixel-accurate monochrome framebuffer of a fixed size (typically 128×64), edited as a canvas of typed objects: text, lines, rectangles, bitmap images, special elements.



LCD editor. Left: canvas tools and object types. Center: an exact 128×64 preview. Right: linking the screen to FSM states and layers.

- **Select / Add text / Line / Rectangle / Invert row / Special element / Glyph editor** — the core drawing tools; every object stays editable (never rasterized) until export.
- **Pixel editor** becomes available for a selected bitmap object — point-precise pixel editing.
- Each screen can be linked to several FSM states at once and to a layer (such as “Demo” or a custom one) through the **Linked FSM states** panel on the right.

Raster Import

Turn finished artwork (a logo, an icon, a scan) into a 1bpp image and keep editing it pixel-by-pixel as a regular bitmap layer.

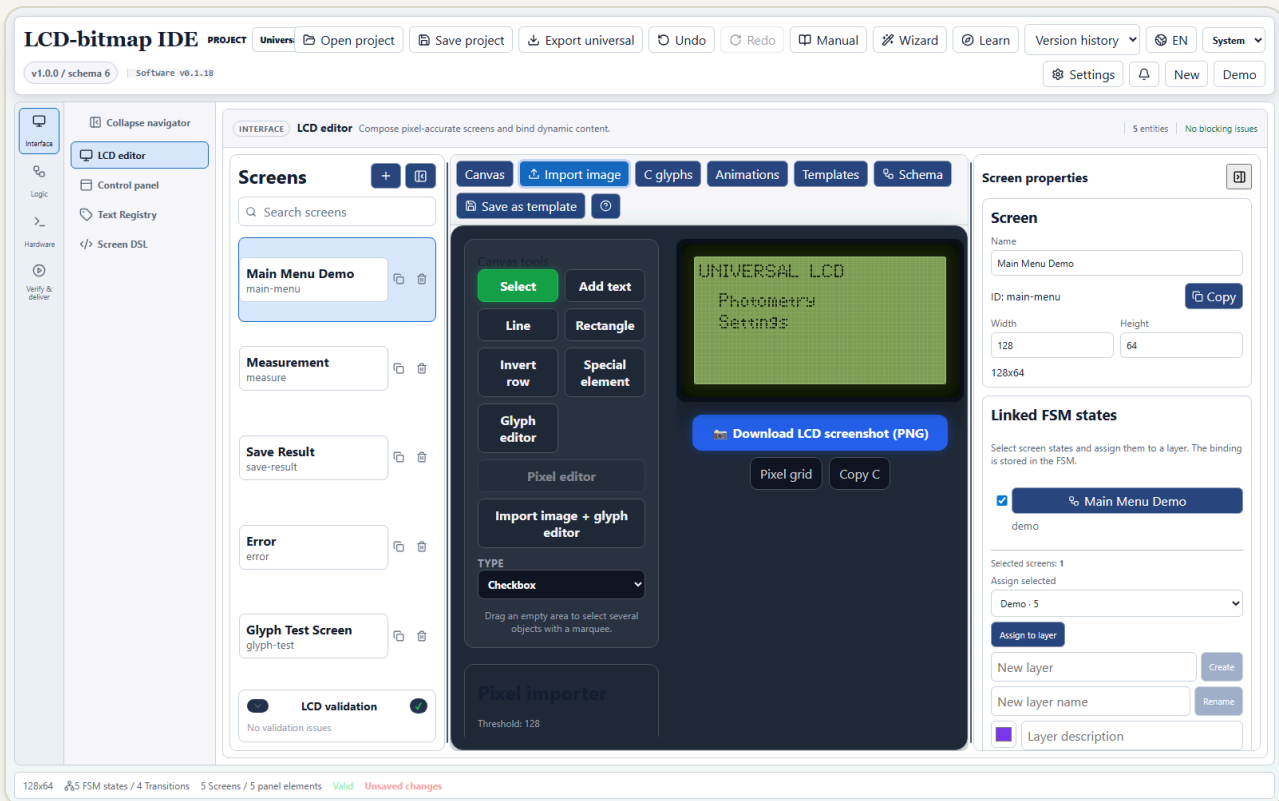
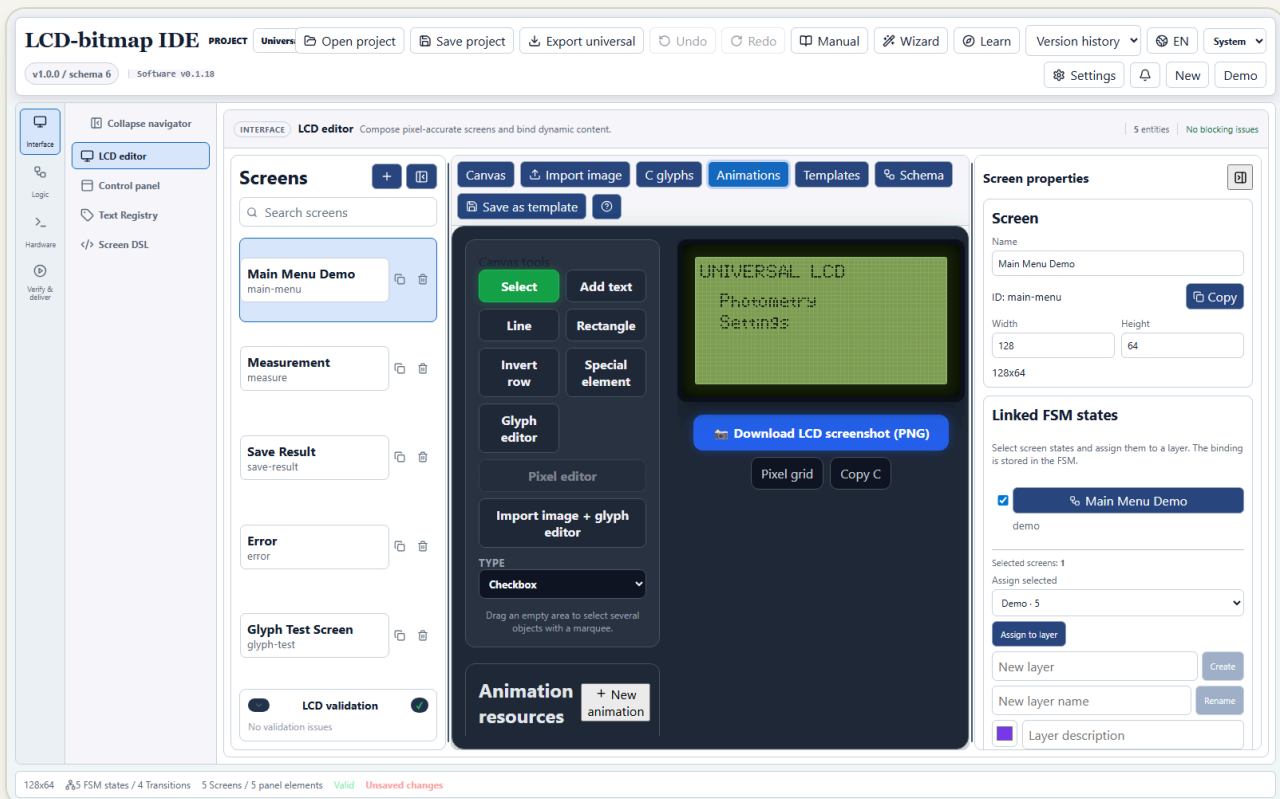


Image import. The **Import image** tab in the LCD editor: PNG/JPEG/BMP/SVG → threshold + dithering → 1bpp.

- 1 Open the screen you want the image placed on, then switch to the **Import image** tab.
- 2 Choose a file and adjust **Threshold** and **Dithering** until the 128×64 preview reads clearly.
- 3 Click **Insert and edit bitmap** — the image is inserted on the screen, selected, and the pixel editor opens for fine-tuning right away.

Animations

An animation resource is an ordered set of 1bpp frames, each with its own duration and a shared loop flag. One resource can be bound either to an entire screen or to a single bitmap layer, leaving the rest of the screen static.



Animations panel. The **Animations** tab sits next to **Import image** – the same conversion pipeline creates animation frames.

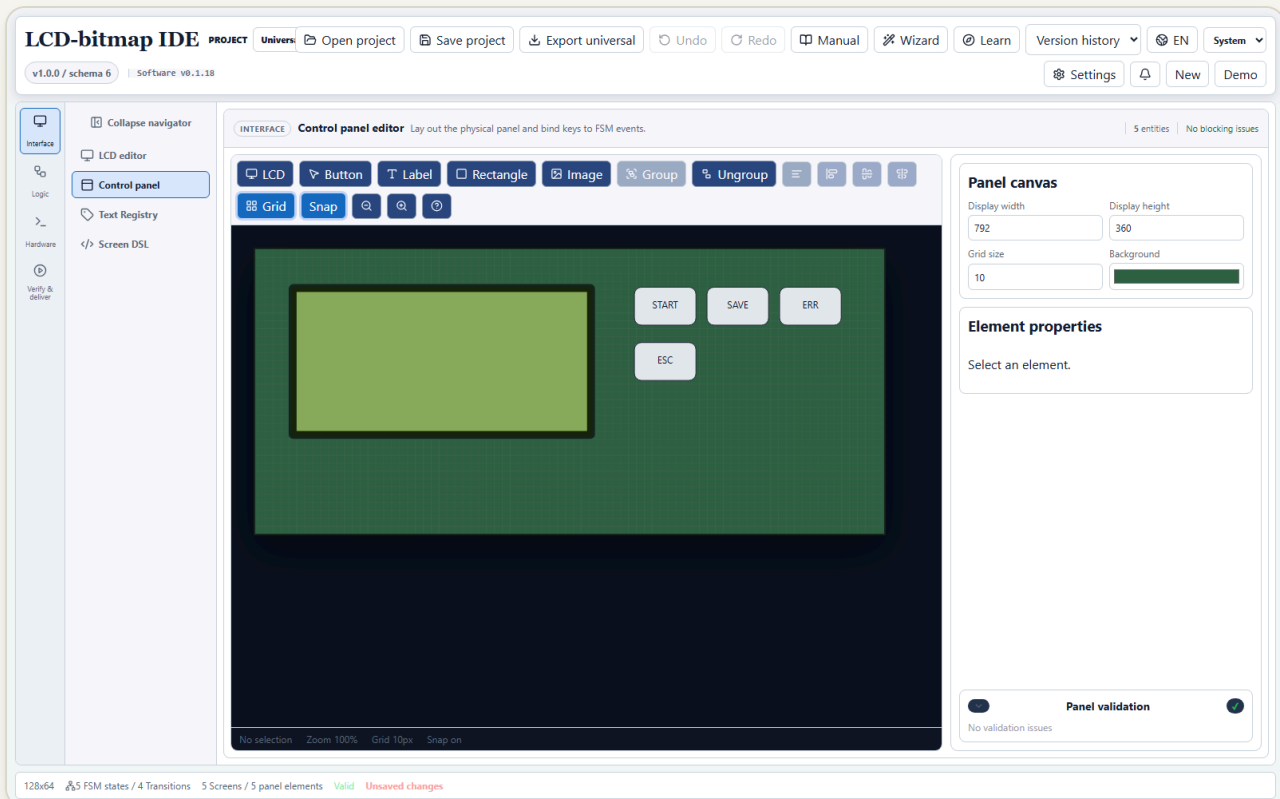
- 1 Click **New animation** and pick an image for the first frame — sized to the selected bitmap object, or to the screen if nothing is selected.
- 2 Add more frames from other images, tune each frame's duration in milliseconds, and duplicate, remove or reorder them.
- 3 Turn on **Loop** for a repeating animation (a busy indicator, say) or leave it off for a one-shot sequence.
- 4 Preview the result with play / replay / the scrubber on a deterministic clock, then bind the resource with **Bind to screen** (whole screen) or **Bind to selected layer** (only the selected bitmap object).

For changing numeric values (a countdown timer, say) use a text field bound to a runtime tag rather than an animation: animation is meant for graphics, not for hundreds of near-identical numeral frames.

On firmware export, every resource becomes a `lcd_animation_frame_t` frame array (bytes, byte count, duration) wrapped in an `lcd_animation_t` (frame table, frame count, `loop` flag) — screen and layer binding metadata is exported alongside it.

Control Panel

The physical layout of buttons and indicators is designed separately from the logic and linked to FSM events.

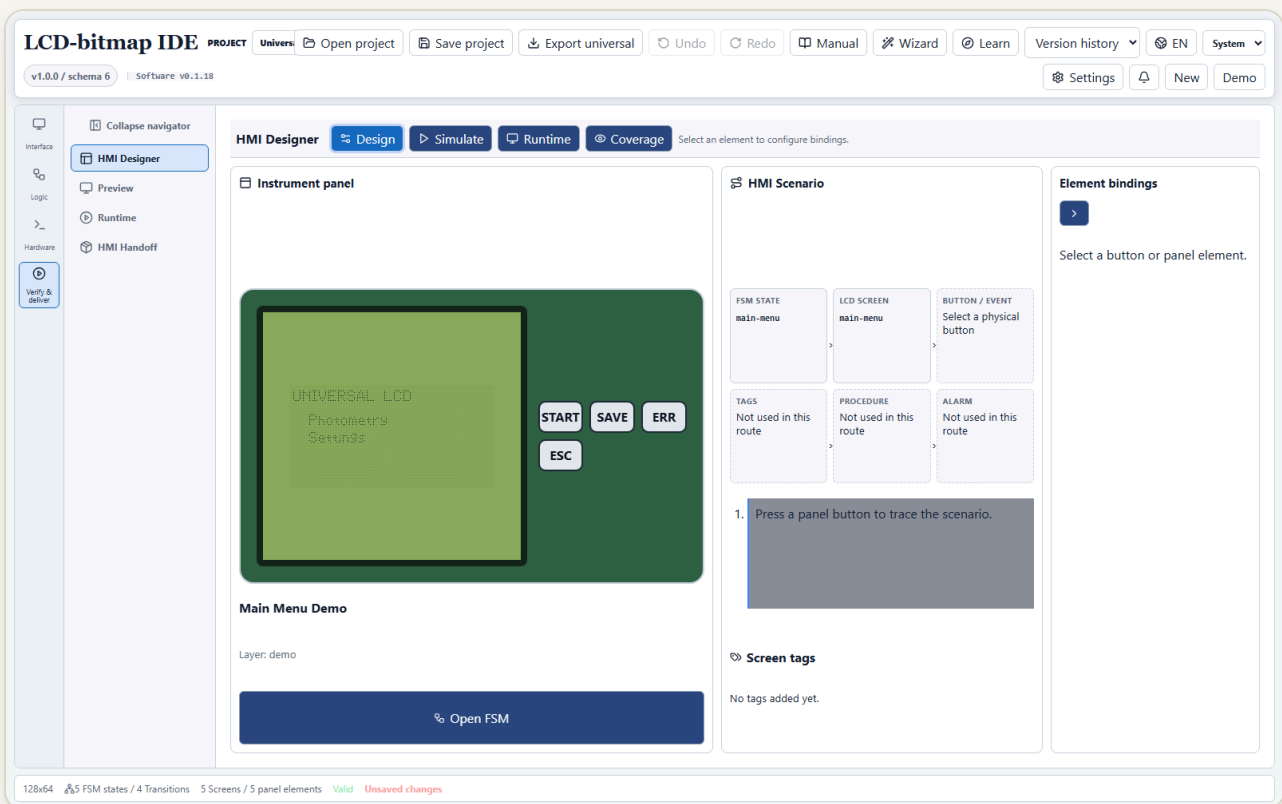


Control panel editor. The **LCD**, **Button**, **Label**, **Rectangle**, **Image** tools let you assemble an exact enclosure layout; every button is linked to an FSM event.

- **Group / Ungroup** and the alignment tools help lay out many elements neatly.
- **Grid** and **Snap** give precise positioning against the enclosure's pixel grid.
- The **Panel validation** panel at the bottom right immediately flags any button with no linked event.

HMI Designer

Combines the screen and the physical panel into one scene to check the whole interaction scenario at once: which button does what on which screen, and which tags and procedures are involved.



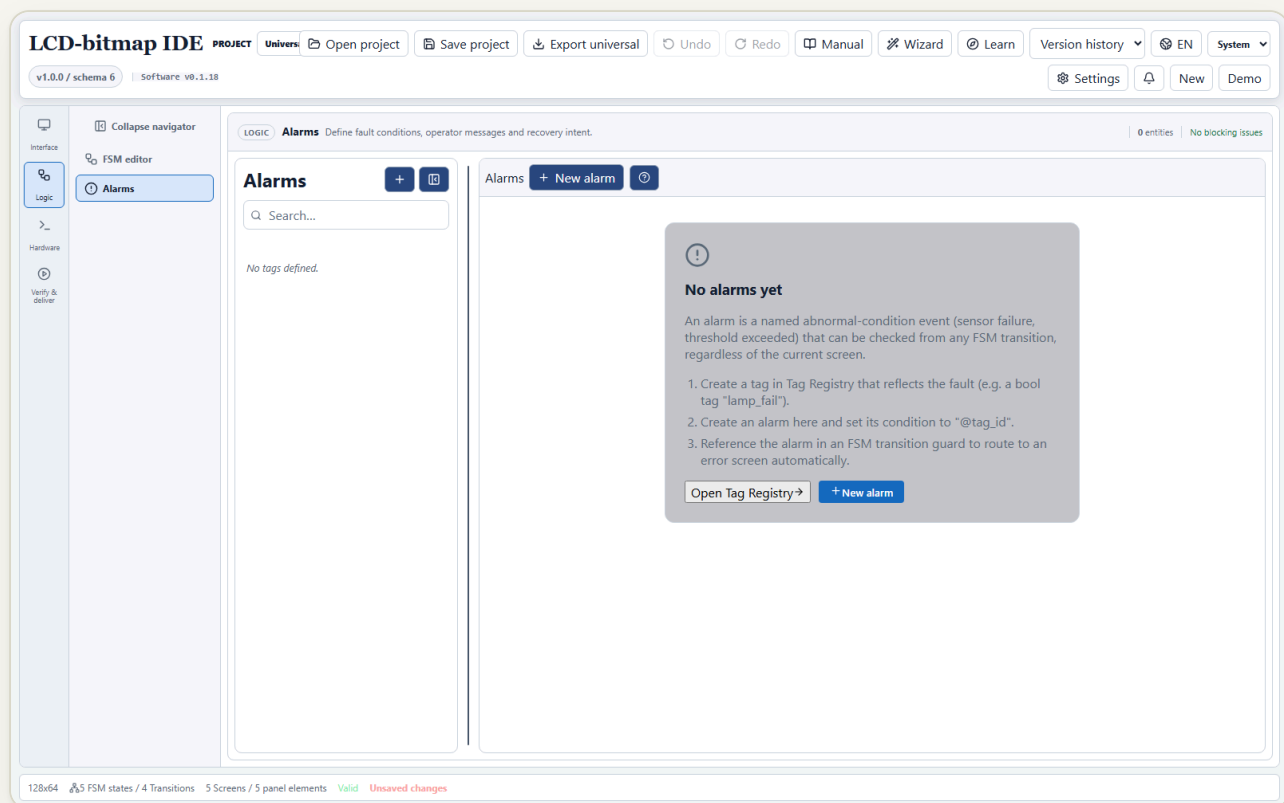
HMI Designer. Modes **Design** / **Simulate** / **Runtime** / **Coverage**; the right column traces which state, screen, button, tag, procedure and alarm are involved in the current scenario.

Coverage mode helps find states or panel elements never used in any scenario — useful before handing the project off to firmware.

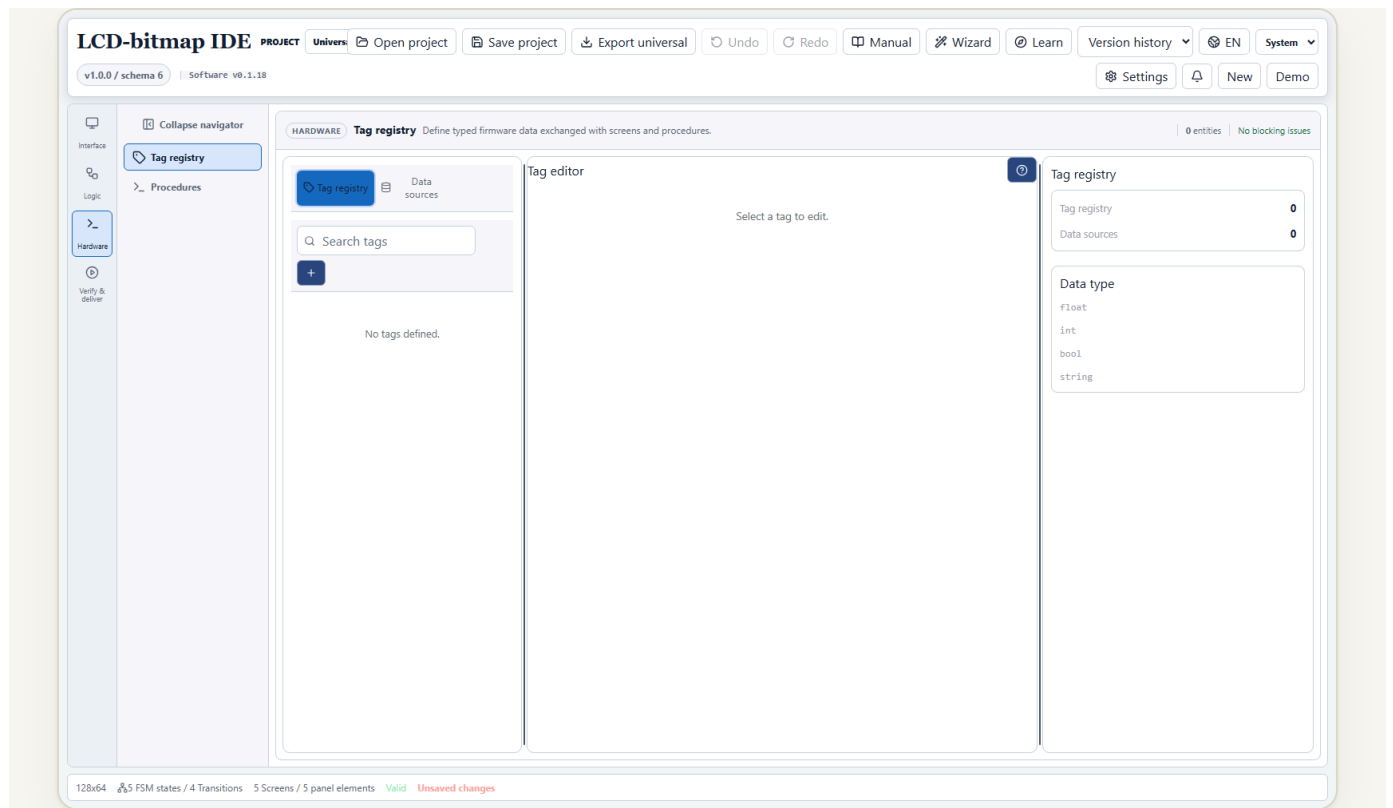
Tags, Procedures, Alarms

Three linked registries describe what physically happens on the device without being drawn on a screen directly.

SECTION	PURPOSE	WHERE
Tag registry	Typed firmware data (float / int / bool / string) exchanged between screens and procedures	Hardware → Tag registry
Procedures	Named backend processes (calibration, say), documented with engineering notes	Hardware → Procedures
Alarms	Named abnormal-condition events, checkable from any FSM transition regardless of the current screen	Logic → Alarms



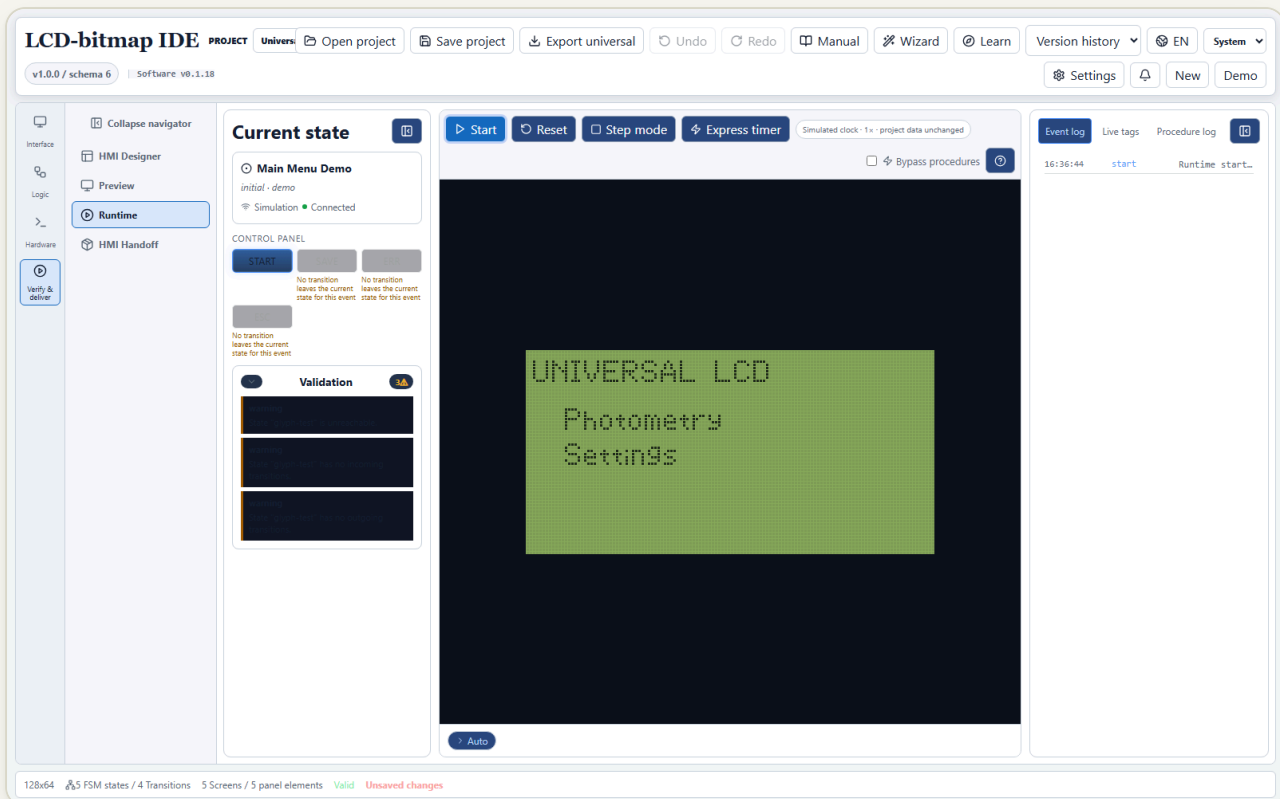
Creating an alarm. The built-in hint spells out the exact sequence: a tag in the registry → an alarm with condition `@tag_id` → a reference to it in an FSM transition guard.



Tag registry. The right column lists the supported data types for new tags.

Runtime Preview

Run the FSM logic without firmware: press virtual buttons, watch a live render of the LCD screen and the event log — before handing the project to the embedded team.

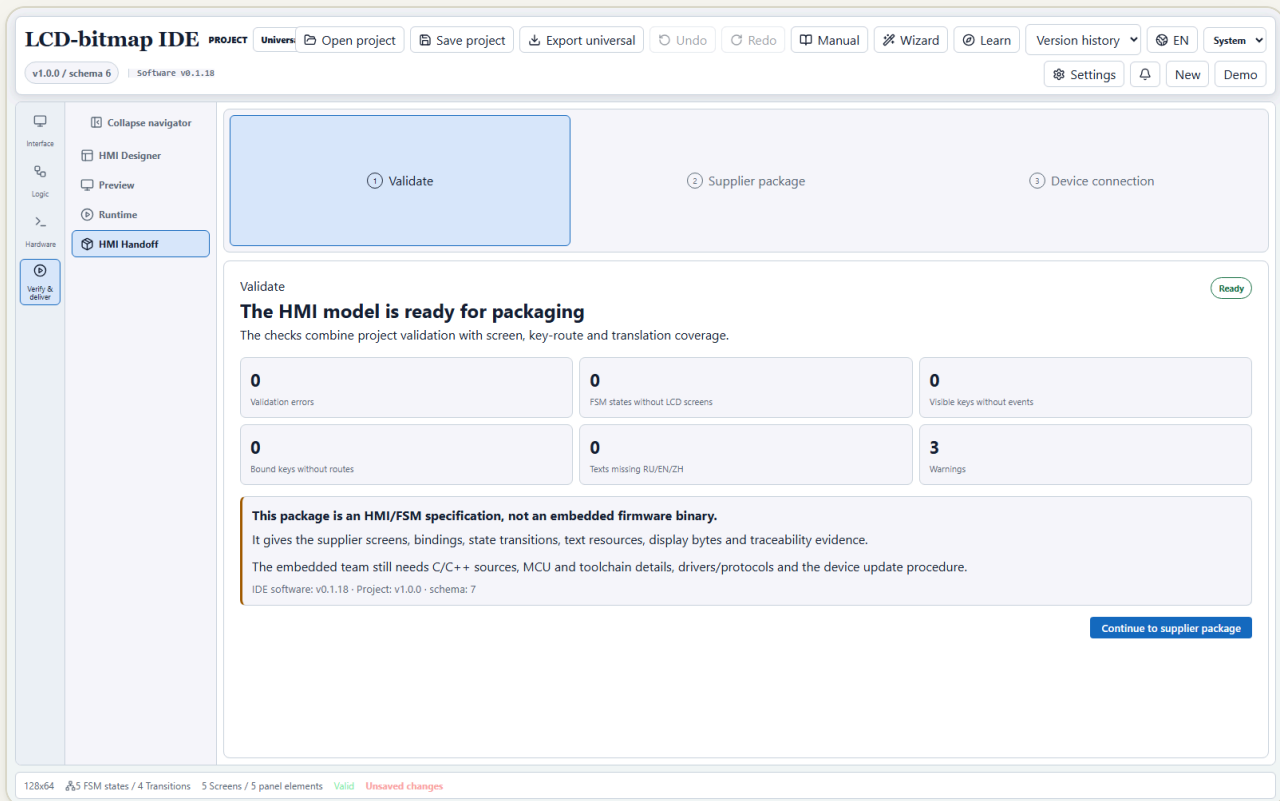


Runtime preview. Left: the current state and the buttons available from it (unavailable ones are dimmed with a reason). Center: an exact screen render. Right: the event, tag and procedure log.

- **Step mode** advances one event at a time — useful for tracing complex logic.
- **Express timer** speeds up simulated timers without changing transition logic.
- **Bypass procedures** lets you skip external backend procedures while running the scenario end to end.
- The **Validation** panel on the left shows the same warnings the project validator does — an unreachable state, for instance.

Firmware Handoff

The final step before release: assembling a package for the embedded team
— validation, a supplier package, a device-connection check.

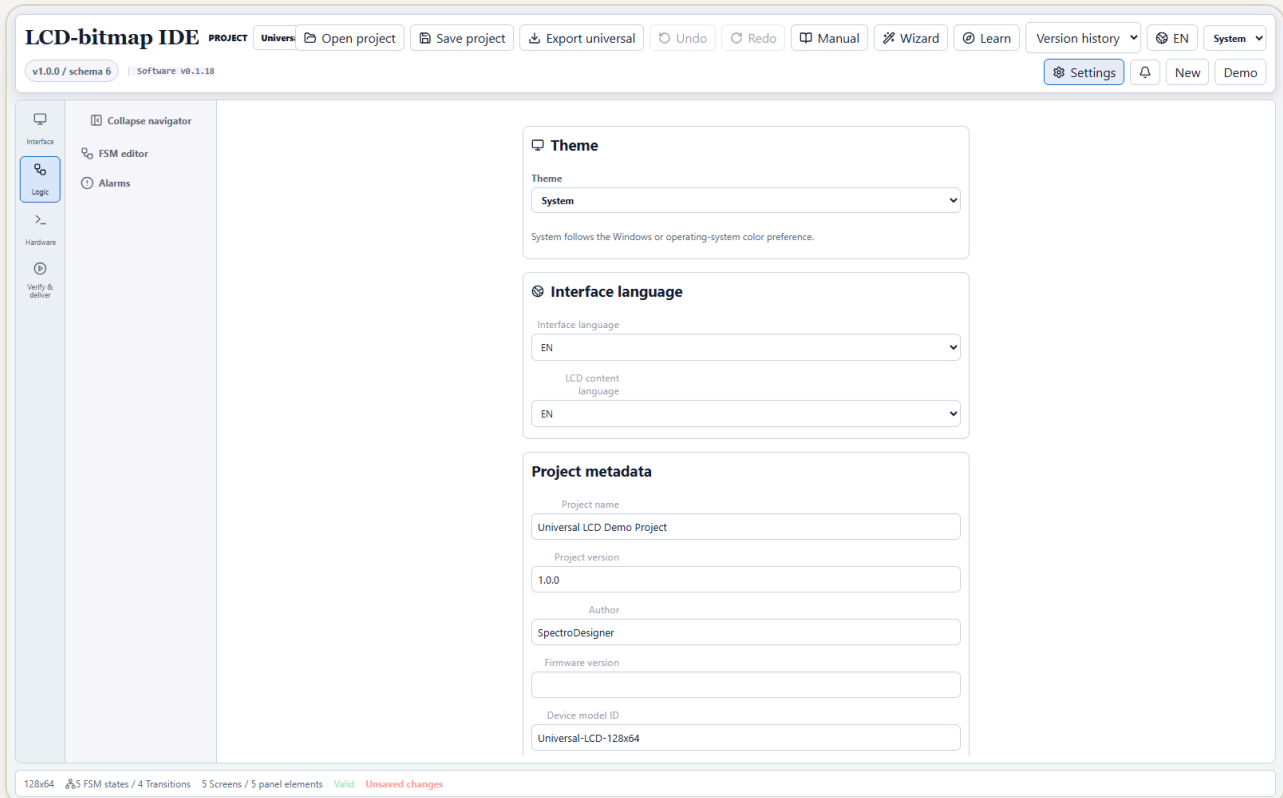


HMI Handoff → Validate. The summary combines the overall project validation with coverage checks for screens, button routes and interface-language translations.

Important: this package is an HMI/FSM specification, not a firmware binary. It hands the supplier screens, bindings, state transitions, text resources and display bytes with traceability. C/C++ sources, MCU and toolchain details, drivers and protocols, and the device update procedure remain the embedded team's responsibility.

API and MCP Servers

The Electron and Tauri builds run the identical local automation contract: REST on `127.0.0.1:8766` and MCP (JSON-RPC over HTTP) on `127.0.0.1:8767/mcp` — letting external scripts, SCADA systems and LLM agents read and edit the open project directly.



Settings → API & MCP servers. Desktop builds show live status for both servers, their addresses and whether token verification is enabled; unavailable in the web build.

- Both servers bind only to `127.0.0.1` and never listen on an external interface.
- `GET /health` on the MCP port is an availability check with no token and no project access.
- The `LCD_IDE_AUTOMATION_TOKEN` environment variable turns on mandatory Bearer authorization for write operations.
- The full tool list, schemas and integration examples live in [docs/API_MCP_CONNECTORS.md](https://docs.lcd-bitmap.com/API_MCP_CONNECTORS.md).

Troubleshooting

SYMPTOM	CAUSE	FIX
“Add state” is disabled	The FSM graph is open read-only	Turn on <code>Edit graph</code> in the FSM toolbar
Transitions disappeared from the canvas	Overview, a subsystem focus or a layer filter is active	Check the “shown / total” counter next to the filters and click Reset filters
<code>connection refused</code> on 8766/8767	The desktop app is not running, or another process holds the port	Start the Electron/Tauri build and check its automation console
The API returns <code>project: null</code>	No project is open	Open a <code>.lcdproj</code> file or the demo project in the app
An MCP/REST request does not respond	The renderer did not answer within 5 seconds	Make sure the app window is open and not blocked by a modal dialog

LCD-bitmap IDE · version 0.1.18 · Electron and Tauri builds share one automation contract and one project model. Detailed technical documents live in the repository’s docs/ directory.